数据结构 DataStructure/

> 约定：默认 **1-indexed**（构造传点数/元素数 `n`，内部开 `n+1`）。 依赖 `Template/ihxnan.hpp` 的类型别名（`ll`、`vi`、`pii` 等）；`SegmentTree` 还依赖宏 `ls` / `rs`。

DSU 并查集 `DSU.hpp`

- **用法**：`DSU dsu(n)`（点号 `1..n`）；`merge(x, y)` 合并，返回是否真的合并（可用于 Kruskal 计数）；`same(x, y)` 判连通；`size(x)` 返回 `x` 所在连通块大小。
- **实现**：路径减半 + 按大小合并，均摊近 `O(α)`。
- **注意**：下标 `0` 也存在（自根），但按约定从 `1` 开始使用。

```
struct DSU
{
    vector<int> fa, sz; // fa[i]: i 的父节点（根满足 fa[i]==i）；sz[i]: i 为根时连通块大小

    // 构造 n 个点（编号 1..n，内部开 n+1），每个点先各自成根
    DSU(int n) : fa(n + 1), sz(n + 1, 1)
    {
        iota(fa.begin(), fa.end(), 0); // fa[i] = i，初始自为根
    }

    // 查根（路径减半：边走边把 fa 指向祖父，均摊近 O(α)）
    int get(int x)
    {
        while (x != fa[x])
            x = fa[x] = fa[fa[x]]; // 压缩：把当前点直接连到祖父
        return x;
    }

    // 合并 x、y 所在集合，返回是否真的合并（同集合返回 false）
    bool merge(int x, int y)
    {
        if ((x = get(x)) == (y = get(y))) // 先各取根
            return false;
        fa[y] = x;      // 小根挂到大根下（按大小合并，树高 O(log n)）
        sz[x] += sz[y]; // 更新合并后的集合大小
        return true;    // 可用于 Kruskal 统计合并次数
    }

    // x、y 是否在同一集合
    bool same(int x, int y)
    {
        return get(x) == get(y);
    }

    int size(int x) // x 所在连通块大小（取根后读 sz）
    {
        return sz[get(x)];
    }
};
```

树状数组 Fenwick.hpp

- **用法**：`Fenwick<ll> fw(n)`；`add(x, v)` 单点加；`query(x)` 前缀和 `[1..x]`；`query(l, r)` 区间和；`select(k)` 树上二分。
- **复杂度**：单次 $O(\log n)$。
- **select**：返回满足"前缀和 `<= k`"的最大下标 `x`（配合"每个位置 `add(pos, 1)`"可查第 `k` 小：答案为 `select(k - 1) + 1`）。

```
template <typename T> struct Fenwick
{
    int n;
    vector<T> tree; // tree[i] 保存区间 (i-lowbit(i), i] 的区间和

    // 构造大小为 n 的树状数组（下标 1..n）
    Fenwick(int n) : n(n), tree(n + 1)
    {
    }

    // 单点加：位置 x 增加 v
    void add(int x, const T &v)
    {
        for (; x ≤ n; x += x & -x) // x += lowbit(x)：逐层跳到"管辖 x 的更高点"
            tree[x] += v;
    }

    // 前缀和 [1..x]
    T query(int x)
    {
        T ans{};                    // 值初始化为 T 的零元
        for (; x > 0; x -= x & -x) // x -= lowbit(x)：逐层剥掉最低二进制位
            ans += tree[x];
        return ans;
    }

    // 区间和 [l..r] = 前缀(r) - 前缀(l-1)
    T query(int l, int r)
    {
        return query(r) - query(l - 1);
    }

    // 树上二分：返回满足"前缀和 ≤ k"的最大 x（配合每位置 add(pos,1) 查第 k 小）
    int select(const T &k)
    {
        int x = 0;
        T cur{};
        for (int i = 1 << __lg(n); i; i >>= 1) // 从最高二进制位往下试跳
            if (x + i ≤ n && cur + tree[x + i] ≤ k)
                cur += tree[x += i]; // 前缀和仍 ≤ k，就跳过去
        return x;
    }
};
```

线段树（区间加 / 区间求和）SegmentTree.hpp

- **用法**：SegmentTree<ll> sg(n) 或 SegmentTree<ll> sg(arr)（arr 为 1-indexed、长度 n+1）；update(l, r, k) 区间加 k；query(l, r) 区间和；重建可调 build()。
- **复杂度**：单次 O(log n)。
- **注意**：**依赖骨架宏 ls / rs**；T 需支持 +、+=，且懒标记 0 表示无标记；区间长用 (r - l + 1) * k，注意溢出（建议 T = ll）。

```
template <class T> struct SegmentTree
{
    int n;
    vector<T> in;          // 原数组（1-indexed）
    vector<T> tree, tag;   // tree[p]: 节点值（区间和）；tag[p]: 区间加懒标记（0 表示无）

    SegmentTree(int n) : n(n), in(n + 1), tree(4 * n), tag(4 * n)
    {
    }

    SegmentTree(vector<T> &arr) : n(arr.size() - 1), in(arr), tree(4 * n), tag(4 * n)
    {
        build(); // 直接从数组建树
    }

    // 由两个儿子合并出父亲（此处为区间和：左 + 右）
    void push_up(int p)
    {
        tree[p] = tree[ls(p)] + tree[rs(p)];
    }

    void build(int p, int l, int r) // 递归建树：节点 p 覆盖 [l, r]
    {
        if (l == r) // 叶子：直接取原数组值
        {
            tree[p] = in[l];
            return;
        }
        int mid = l + r >> 1;
        build(ls(p), l, mid);
        build(rs(p), mid + 1, r);
        push_up(p);
    }

    // 给 p 覆盖的整段 [l, r] 整体加 k，并打懒标记（O(1)）
    void change(int p, int l, int r, T k)
    {
        tree[p] += (r - l + 1) * k; // 区间和增加 长度 * k
        tag[p] += k;                // 懒标记累加，留待访问儿子时再下传
    }

    // 把 p 的懒标记下传给孩子（保证访问孩子前其值正确）
    void push_down(int p, int l, int r)
    {
        if (tag[p]) // 有标记才下传
        {
            int mid = l + r >> 1;
            change(ls(p), l, mid, tag[p]);
            change(rs(p), mid + 1, r, tag[p]);
            tag[p] = 0; // 清空本节点标记
        }
    }

    // 递归修改：把 [ul, ur] 整体加 k
```

```
    void update(int ul, int ur, int p, int l, int r, T k)
    {
        if (ul ⩽ l && r ⩽ ur) // 当前区间被 [ul, ur] 完全覆盖
        {
            change(p, l, r, k);
            return;
        }
        push_down(p, l, r); // 未完全覆盖，先下传再分别递归左右
        int mid = l + r >> 1;
        if (ul ⩽ mid)
            update(ul, ur, ls(p), l, mid, k);
        if (ur > mid)
            update(ul, ur, rs(p), mid + 1, r, k);
        push_up(p);
    }

    // 递归查询 [ql, qr] 的和
    T query(int ql, int qr, int p, int l, int r)
    {
        if (ql ⩽ l && r ⩽ qr) // 完全覆盖，直接返回节点值
            return tree[p];
        push_down(p, l, r);
        T ans = 0;
        int mid = l + r >> 1;
        if (ql ⩽ mid)
            ans += query(ql, qr, ls(p), l, mid);
        if (qr > mid)
            ans += query(ql, qr, rs(p), mid + 1, r);
        return ans;
    }

    // ---- 对外接口（以整棵树 [1, n] 为范围）----
    void build()
    {
        build(1, 1, n);
    }

    void update(int ul, int ur, T k) // [ul, ur] 区间加 k
    {
        update(ul, ur, 1, 1, n, k);
    }

    T query(int ql, int qr) // 查询 [ql, qr] 区间和
    {
        return query(ql, qr, 1, 1, n);
    }
};
```

ST 表（稀疏表） ST.hpp

- **用法**：`ST st(n)` 或 `ST st(arr)`（1-indexed、长度 `n+1`）；先 `init()`，再 `query(l, r)`。
- **复杂度**：预处理 `O(n log n)`，查询 `O(1)`。
- **注意**：仅适用于**可重复结合**（幂等）运算，默认取 `max`，可传自定义二元函数（如 `gcd`）；只支持 `int`。

```
struct ST
{
    const int n, len;                    // n: 元素个数; len = floor(log2 n)+1 层
    vector<int> in;                      // 原数组 (1-indexed)
    vector<vector<int>> st;              // st[i][j]: 区间 [j, j+2^i-1] 的答案
    function<int(int, int)> Max;         // 可重复结合的合并函数 (默认取 max)

    ST(
        int n, function<int(int, int)> Max = [](int a, int b) { return max(a, b); })
        : n(n), len(__lg(n) + 1), in(n + 1), Max(Max) // len 层足够覆盖任意长度
    {
        st.resize(len, vector<int>(n + 1));
    }

    ST(
        vector<int> &arr, function<int(int, int)> Max = [](int a, int b) { return max(a, b); })
        : n(arr.size() - 1), len(__lg(n) + 1), in(arr), Max(Max)
    {
        st.resize(len, vector<int>(n + 1));
    }

    // 预处理: 倍增递推, st[i][j] = Max(前半段, 后半段)
    void init()
    {
        for (int i = 1; i <= n; ++i)
            st[0][i] = in[i]; // 第 0 层: 长度为 1 的区间
        for (int i = 1; i < len; ++i)
            for (int j = 1; j + (1 << i) - 1 <= n; ++j)
                st[i][j] = Max(st[i - 1][j], st[i - 1][j + (1 << i - 1)]); // 两段 2^(i-1) 拼成 2^i
    }

    // O(1) 查询 [l, r]: 用两段长度 2^k 的区间覆盖 (可重叠, 故运算需幂等)
    int query(int l, int r)
    {
        int k = __lg(r - l + 1); // 不超过区间长度的最大 2 的幂
        return Max(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

整数位封装 Bit.hpp

- **用法**：bit<unsigned long long> b(x)；把整数的位运算封装成成员函数 / 运算符，可读性更好。
- **注意**：位下标从 LSB（第 0 位）开始；T 需为无符号整型（unsigned / unsigned long long 等），避免右移补符号位。小规模（位数少）直接裸位运算更快。

```
template <class T> struct bit // T 需为无符号整型（如 unsigned long long）
{
    T value;                                                    // 被封装的整数
    bit(T x = 0) : value(x) {}                                  // 构造（默认 0）
    // ---- 单比特操作（位下标从 LSB 第 0 位起）----
    bool operator[](int x) const { return value >> x & 1; }  // 读第 x 位
    void set(int x) { value ⊨ T(1) << x; }                      // 置第 x 位为 1
    void reset(int x) { value &= ~(T(1) << x); }                // 置第 x 位为 0
    void flip(int x) { value ^= T(1) << x; }                    // 翻转第 x 位
    // ---- 移位（返回新 bit，不修改自身）----
    bit operator<<(int x) const { return bit(value << x); }
    bit operator>>(int x) const { return bit(value >> x); }
    bit &operator⇐(int x) { value ⇐ x; return *this; }
    bit &operator⇒(int x) { value ⇒ x; return *this; }
    // ---- 按位运算 ----
    bit operator&(const bit &o) const { return bit(value & o.value); }
    bit operator|(const bit &o) const { return bit(value | o.value); }
    bit operator^(const bit &o) const { return bit(value ^ o.value); }
    bit &operator&=(const bit &o) { value &= o.value; return *this; }
    bit &operator⊨(const bit &o) { value ⊨ o.value; return *this; }
    bit &operator^=(const bit &o) { value ^= o.value; return *this; }
    bit operator~() const { return bit(~value); }               // 按位取反
    // ---- 比较 ----
    bool operator⩵(const bit &o) const { return value ⩵ o.value; }
    bool operator≠(const bit &o) const { return value ≠ o.value; }
    bool operator<(const bit &o) const { return value < o.value; }
    bool operator>(const bit &o) const { return value > o.value; }
    bool operator⩽(const bit &o) const { return value ⩽ o.value; }
    bool operator⩾(const bit &o) const { return value ⩾ o.value; }
};
```

FHQ Treap（数组版，按值 / 按大小分裂） Treap.hpp

- **用法**：Treap<int> tp(节点数上限)；insert(x) 插入；erase(x) 删除一个 x；rank(x) 比 x 小的元素个数 + 1；val(k) 第 k 小；pre(x) 前驱（< x 的最大值）；back(x) 后继（> x 的最小值）；count(x) x 的个数；size() 元素总数。
- **复杂度**：单次期望 O(log n)。
- **注意**：构造参数为**节点数上限**（内部开 n+1，每次 insert 新建一个节点）；split_val 的 eq 参数控制 == k 归左还是归右；指针版见 TreapPtr。

```
template <class T> struct Treap
{
    struct Node
    {
        T val;
        int key, siz, ch[2]; // key: 随机优先级（key 小者作根）；siz: 子树大小；ch[0/1]: 左右儿子
        void clear()
        {
            ch[0] = ch[1] = siz = key = 0;
            val = T{};
        }
    };

    int cnt, root;
    vector<Node> t;
    mt19937_64 gen;

    Treap(int n) : cnt(0), root(0), t(n + 1), gen(chrono::steady_clock::now().time_since_epoch().count
())
    {
    }

    int rnd()
    {
        return int(gen());
    }

    // 用儿子更新自己的子树大小
    void update(int now)
    {
        t[now].siz = t[t[now].ch[0]].siz + t[t[now].ch[1]].siz + 1;
    }

    // 合并两棵树（要求 u 中所有值 ≤ v 中所有值），按随机优先级 key 确定新根（key 小者作根）
    int merge(int u, int v)
    {
        if (!u || !v)
            return u | v;             // 有一棵为空：返回另一棵
        if (t[u].key < t[v].key)      // u 优先级更高，u 作根
        {
            t[u].ch[1] = merge(t[u].ch[1], v); // v 接到 u 右子树
            update(u);
            return u;
        }
        else                          // v 优先级更高，v 作根
        {
            t[v].ch[0] = merge(u, t[v].ch[0]); // u 接到 v 左子树
            update(v);
            return v;
        }
    }

    // 按值分裂：值 < k（eq 时含 == k）的节点进 x，其余进 y
    void split_val(int now, const T &k, int &x, int &y, bool eq = true)
```

```
{
    if (!now)
        return (void)(x = y = 0);
    if (t[now].val < k || (eq && t[now].val == k))
        x = now, split_val(t[now].ch[1], k, t[now].ch[1], y, eq); // now 归 x，继续分右子树
    else
        y = now, split_val(t[now].ch[0], k, x, t[now].ch[0], eq);  // now 归 y，继续分左子树
    update(now); // 回溯时更新子树大小
}

// 按名次分裂：前 k 小（含）进 x，其余进 y
void split_k(int now, int k, int &x, int &y)
{
    if (!now)
        return (void)(x = y = 0);
    if (t[t[now].ch[0]].siz < k) // 左子树不够 k 个，now 及左子树归 x
        x = now, split_k(t[now].ch[1], k - t[t[now].ch[0]].siz - 1, t[now].ch[1], y);
    else                         // 左子树已够 k 个，now 归 y
        y = now, split_k(t[now].ch[0], k, x, t[now].ch[0]);
    update(now); // 回溯时更新子树大小
}

// 取第 k 小（并合并回 now）
T kth(int &now, int k)
{
    int a, b, c, d;
    split_k(now, k - 1, a, b);
    split_k(b, 1, c, d);
    T res = t[c].val;
    now = merge(a, merge(c, d));
    return res;
}

void insert(const T &x) // 新建节点，按值分裂后插入中间
{
    int u = ++cnt, a, b;
    t[u].val = x, t[u].key = rnd(), t[u].siz = 1;
    split_val(root, x, a, b);
    root = merge(merge(a, u), b);
}

void erase(const T &x) // 删除一个 x（不存在则无操作）
{
    int a, b, c, d;
    split_val(root, x, a, b, false); // a: < x
    split_k(b, 1, c, d);             // c: 值为 x 的一个节点
    if (c && t[c].val == x)
        t[c].clear(), root = merge(a, d);
    else
        root = merge(a, merge(c, d));
}

int size() // 元素总数
{
    return t[root].siz;
}

int count(const T &x) // x 出现次数
{
    int a, b, c, d;
    split_val(root, x, a, b, false); // a: < x
    split_val(b, x, c, d, true);     // c: == x
    int res = t[c].siz;
    root = merge(a, merge(c, d));
```

```
        return res;
    }

    int rank(const T &x) // x 的排名（比它小的个数 + 1）
    {
        int a, b;
        split_val(root, x, a, b, false);
        int res = t[a].siz + 1;
        root = merge(a, b);
        return res;
    }

    T val(int k) // 第 k 小
    {
        return kth(root, k);
    }

    T pre(const T &x) // 前驱（< x 的最大值）
    {
        int a, b;
        split_val(root, x, a, b, false);
        T res = kth(a, t[a].siz);
        root = merge(a, b);
        return res;
    }

    T back(const T &x) // 后继（> x 的最小值）
    {
        int a, b;
        split_val(root, x, a, b);
        T res = kth(b, 1);
        root = merge(a, b);
        return res;
    }
};
```

FHQ Treap（指针版，免预估节点数）`TreapPtr.hpp`

- **用法**：与 `Treap.hpp` 完全一致的 API；`TreapPtr<int> tp;` 直接用，无需估计节点数上限。
- **实现**：内存池 `pool` 统一管理新节点，删除的节点进回收站 `trash` 复用，析构时统一释放。
- **注意**：禁用了拷贝（避免双重释放）；`kth` 对空树返回 `T{}`。

```
template <class T> struct TreapPtr
{
    struct Node
    {
        T val;
        int key, siz;
        Node *ch[2];
        Node(const T &v = T{}, int k = 0) : val(v), key(k), siz(1)
        {
            ch[0] = ch[1] = nullptr;
        }
    };

    Node *root;
    vector<Node *> pool, trash; // 内存池 / 回收站
    mt19937_64 gen;

    TreapPtr() : root(nullptr), gen(chrono::steady_clock::now().time_since_epoch().count()) {}
    TreapPtr(const TreapPtr &) = delete;            // 禁止拷贝
    TreapPtr &operator=(const TreapPtr &) = delete;
    ~TreapPtr() // 析构统一释放内存池
    {
        for (Node *p : pool)
            delete p;
    }

    int rnd()
    {
        return int(gen());
    }

    int sz(Node *p) // 空节点大小为 0
    {
        return p ? p->siz : 0;
    }

    // 取新节点：优先复用回收站
    Node *newNode(const T &v)
    {
        Node *p;
        if (!trash.empty())
        {
            p = trash.back(), trash.pop_back();
            p->val = v, p->key = rnd(), p->siz = 1;
            p->ch[0] = p->ch[1] = nullptr;
        }
        else
            pool.push_back(p = new Node(v, rnd()));
        return p;
    }

    void update(Node *now)
    {
        now->siz = sz(now->ch[0]) + sz(now->ch[1]) + 1;
    }
```

```
Node *merge(Node *u, Node *v) // 合并（u 中所有值 ≤ v），按 key 小者作根
{
    if (!u || !v)
        return u ? u : v; // 有一棵为空：返回另一棵
    if (u→key < v→key)  // u 优先级更高，u 作根
    {
        u→ch[1] = merge(u→ch[1], v); // v 接到 u 右子树
        update(u);
        return u;
    }
    else                  // v 优先级更高，v 作根
    {
        v→ch[0] = merge(u, v→ch[0]); // u 接到 v 左子树
        update(v);
        return v;
    }
}

void split_val(Node *now, const T &k, Node *&x, Node *&y, bool eq = true) // 按值分裂
{
    if (!now)
        return (void)(x = y = nullptr);
    if (now→val < k || (eq && now→val == k))
        x = now, split_val(now→ch[1], k, now→ch[1], y, eq); // now 归 x，继续分右子树
    else
        y = now, split_val(now→ch[0], k, x, now→ch[0], eq);  // now 归 y，继续分左子树
    update(now);
}

void split_k(Node *now, int k, Node *&x, Node *&y) // 按名次分裂：前 k 小进 x，其余进 y
{
    if (!now)
        return (void)(x = y = nullptr);
    if (sz(now→ch[0]) < k) // 左子树不够 k 个，now 及左子树归 x
        x = now, split_k(now→ch[1], k - sz(now→ch[0]) - 1, now→ch[1], y);
    else                   // 左子树已够 k 个，now 归 y
        y = now, split_k(now→ch[0], k, x, now→ch[0]);
    update(now);
}

T kth(Node *&now, int k) // 第 k 小
{
    Node *a, *b, *c, *d;
    split_k(now, k - 1, a, b);
    split_k(b, 1, c, d);
    T res = c ? c→val : T{};
    now = merge(a, merge(c, d));
    return res;
}

void insert(const T &x) // 插入
{
    Node *a, *b;
    split_val(root, x, a, b);
    root = merge(merge(a, newNode(x)), b);
}

void erase(const T &x) // 删除一个 x，节点进回收站
{
    Node *a, *b, *c, *d;
    split_val(root, x, a, b, false);
    split_k(b, 1, c, d);
    if (c && c→val == x)
        trash.push_back(c), root = merge(a, d);
```

```
        else
            root = merge(a, merge(c, d));
    }

    int size() // 元素总数
    {
        return sz(root);
    }

    int count(const T &x) // x 出现次数
    {
        Node *a, *b, *c, *d;
        split_val(root, x, a, b, false);
        split_val(b, x, c, d, true);
        int res = sz(c);
        root = merge(a, merge(c, d));
        return res;
    }

    int rank(const T &x) // x 的排名
    {
        Node *a, *b;
        split_val(root, x, a, b, false);
        int res = sz(a) + 1;
        root = merge(a, b);
        return res;
    }

    T val(int k) // 第 k 小
    {
        return kth(root, k);
    }

    T pre(const T &x) // 前驱
    {
        Node *a, *b;
        split_val(root, x, a, b, false);
        T res = kth(a, sz(a));
        root = merge(a, b);
        return res;
    }

    T back(const T &x) // 后继
    {
        Node *a, *b;
        split_val(root, x, a, b);
        T res = kth(b, 1);
        root = merge(a, b);
        return res;
    }
};
```

莫队 Mo.hpp（片段式）

> **片段式**（同 Math/Exgcd.hpp、Graph/Hungarian.hpp）：文件里是若干可直接粘贴的骨架，**不能作为 #include 单独编译**；四段为互斥变体，每次只取其中一段。 约定 **1-indexed**；依赖 ihxnan.hpp 的别名 ti/vi/vl 与分块宏 bi(x)（并要求作用域内存在 int blen，用到 bl/br 时还需 int n）。 共同骨架：查询读进 vector<ti> qry（(l, r, id)）→ 按 bi(l) 分块 + **奇偶优化**排序 → 空窗口 left=1, right=0 四向移动调用 ins/del → ans[id] = 当前贡献。

普通莫队（P2709 / P3901 / P1494）

- **用法**：int blen = sqrt(n) 后排序、开空窗口；只需按题目改写 ins / del 两个 lambda（增删一个下标对答案的贡献）。
- **复杂度**：块长 $\sqrt{n}$，总复杂度 $O((n+m)\sqrt{n})$。
- **注意**：ins / del 以 P2709 的"同色对数"为例；P3901 只需改成维护"种类数"，P1494 改成维护 A, B。

```
int blen = sqrt(n); // 块长一般取 sqrt(n)
vector<ti> qry(m);  // (l, r, id), 1-indexed
int dn = 0;
for (auto &[l, r, id] : qry)
    cin >> l >> r, id = dn++;

sort(qry.begin(), qry.end(), [&](auto &x, auto &y) {
    auto &[l1, r1, id1] = x;
    auto &[l2, r2, id2] = y;
    if (bi(l1) != bi(l2))
        return bi(l1) < bi(l2);
    return bi(l1) & 1 ? r1 > r2 : r1 < r2; // 奇偶优化：块内 r 交替方向
});

vl ans(m);
int left = 1, right = 0; // 空窗口 [1, 0]

// ---- 改这两个 lambda ----
vi cnt(k + 1);
ll res = 0;
auto ins = [&](int x) -> void { res += 2 * ++cnt[arr[x]] - 1; };
auto del = [&](int x) -> void { res -= 2 * cnt[arr[x]]-- - 1; };
// ------------------------

for (auto &[l, r, id] : qry)
{
    while (left < l)
        del(left++);
    while (left > l)
        ins(--left);
    while (right < r)
        ins(++right);
    while (right > r)
        del(right--);
    ans[id] = res;
}
for (auto &p : ans)
    cout << p << endl;
```

莫队 + 值域分块求 mex（P4137）

- **用法**：在普通莫队基础上追加值域块计数 tag[]；ins/del 同时维护 cnt[] 与 tag[bi(值)]（只统计 arr[x] <= n 的值）；回答时先跳到第一个未满的块，再块内线性找第一个未出现的值。
- **复杂度**：查询 $O(\sqrt{n})$ 求 mex，总 $O((n+m)\sqrt{n})$。

- **注意**：值域含 `0`，块长取 `max(2, (int)sqrt(n))` 才能保证 `bi(0) = 0`（值 `0` 归入第 `0` 块）；第 `0` 块比普通块多含一个值 `0`（`+ !idx`）。

```
int blen = max(2, (int)sqrt(n)); // 值域分块需 blen≥2 才能保证 bi(0)=0
int bnum = (n + blen - 1) / blen; // 值域块数

// ... 读入 + 排序 + ins/del 同普通莫队 ...
vi tag(bnum), ans(m), cnt(n + 1);
auto ins = [&](int x) → void {
    if (arr[x] ⩽ n && ++cnt[arr[x]] == 1)
        ++tag[bi(arr[x])];
};
auto del = [&](int x) → void {
    if (arr[x] ⩽ n && !--cnt[arr[x]])
        --tag[bi(arr[x])];
};

// ... 窗口移动后求 mex ...
int idx = 0; // 第一个未满的块（第 0 块含值 0，多一个）
while (tag[idx] == br(idx) - bl(idx) + 1 + !idx)
    ++idx;
idx = bl(idx) - !idx; // 定位到该块左端（第 0 块从 0 开始）
while (cnt[idx])      // 块内找第一个未出现的值
    ++idx;
ans[id] = idx;
```

带修莫队（P1903）

- **用法**：查询是三维 `(l, r, 时间 t)`；修改存 `(pos, after, before)`，用 `front/back` 沿时间轴应用/撤销；块长取 `n^(2/3)`，排序三级（`bi(l)` → 块内 `r` 奇偶 → `bi(l)+bi(r)` 奇偶决定 `t` 方向）。
- **复杂度**：块长 `n^(2/3)`，总 `O(n^(5/3))`。
- **注意**：`front` / `back` 仅当修改落点 `pos` 在当前窗口 `[left, right]` 内时才需改动贡献。

```
int blen = pow(n, 0.66);               // 带修块长 ≈ n^(2/3)
vector<tuple<int, int, int, int>> qry; // (l, r, t, id)
vector<ti> func;                       // (pos, after, before)
int t = 0, dn = 0;
vi tmp = arr; // 记录每次修改的 before
for (int i = 0, l, r; i < m; ++i)
{
    char op;
    cin >> op >> l >> r;
    if (op == 'Q')
        qry.emplace_back(l, r, t, dn++);
    else
        func.emplace_back(l, r, tmp[l]), tmp[l] = r, ++t;
}
sort(qry.begin(), qry.end(), [&](auto &x, auto &y) {
    auto &[l1, r1, t1, id1] = x;
    auto &[l2, r2, t2, id2] = y;
    if (bi(l1) != bi(l2))
        return bi(l1) < bi(l2);
    if (bi(r1) != bi(r2))
        return bi(l1) & 1 ? r1 > r2 : r1 < r2;
    return bi(l1) + bi(r1) & 1 ? t1 > t2 : t1 < t2;
});

// ... ins/del 同普通莫队 ...
auto front = [&](int x) → void { // 时间前进：应用第 x 次修改
    auto &[pos, after, before] = func[x];
    arr[pos] = after;
    if (left ⩽ pos && pos ⩽ right)
```

```
        sum -= !--cnt[before], sum += ++cnt[after] == 1;
};
auto back = [&](int x) -> void { // 时间回退：撤销第 x 次修改
    auto &[pos, after, before] = func[x];
    arr[pos] = before;
    if (left <= pos && pos <= right)
        sum -= !--cnt[after], sum += ++cnt[before] == 1;
};

// ... 四向移动 + 时间轴对齐 ...
while (time < t)
    front(time++);
while (time > t)
    back(--time);
ans[id] = sum;
```

回滚莫队（P14420）

- **用法**：适用于"只有加入好维护、删除难维护"（如区间 max）的情形；右端点在块内只增，左端点临时加入后整体回滚。计数用 unordered_map（值域大）。
- **复杂度**：块长 $\sqrt{n}$，总 $O(n\sqrt{n})$。
- **注意**：del 只回退计数、**不维护 cur**，故进入新块重置窗口后需显式 cur = 0；左端回退目标是"上一块右端之后"，用 br(block - 1) + 1——末块不满时 bl(block) 会越过 n 造成越界。

```
int blen = sqrt(n);
vector<ti> qry(q); // (l, r, id)
int dn = 0;
for (auto &[l, r, id] : qry)
    cin >> l >> r, id = dn++;
sort(qry.begin(), qry.end(), [&](auto &x, auto &y) {
    auto &[l1, r1, id1] = x;
    auto &[l2, r2, id2] = y;
    if (bi(l1) != bi(l2))
        return l1 < l2;
    return r1 < r2; // 同块内 r 升序
});

vl ans(q);
int block = -1;
ll cur = 0, back = 0;
int left = 1, right = 0;
unordered_map<int, int> cnt;
auto ins = [&](int x) -> void { cur = max(cur, arr[x] * ++cnt[arr[x]]); };
auto del = [&](int x) -> void { --cnt[arr[x]]; }; // 只回退计数，不动 cur

for (auto &[l, r, id] : qry)
{
    if (block <= bi(l)) // 进入新块：重置为该块右侧的空窗口
    {
        block = bi(l) + 1;
        while (left < br(block - 1) + 1) // 末块不满时 bl(block) 越界，用 br+1
            del(left++);
        while (right < br(block - 1))
            ins(++right);
        while (right > br(block - 1))
            del(right--);
        cur = 0;
    }
    if (bi(l) == bi(r)) // 同块：暴力扫
    {
        ll res = 0;
        for (int i = l; i <= r; ++i)
```

```
            res = max(res, arr[i] * ++cnt[arr[i]]);
        for (int i = l; i <= r; ++i)
            --cnt[arr[i]];
        ans[id] = res;
    }
    else // 跨块: 右端只增, 左端临时加入后回滚
    {
        while (right < r)
            ins(++right);
        back = cur;
        while (left > l)
            ins(--left);
        ans[id] = cur;
        while (left < br(block - 1) + 1)
            del(left++);
        cur = back;
    }
}
for (auto &p : ans)
    cout << p << endl;
```